About this PDF

Built only from: **Artificial Intelligence and Machine Learning (Ensemble Techniques and Unsupervised Learning)** — https://nptel.ac.in/courses/106105077

Licence: CC BY-SA 4.0. Accessed .

Covers 6 of 6 syllabus sub-topics of this unit.

Combining multiple learners: Model combination schemes, Voting

Definition

Combining multiple individual learners into a combined predictive model leverages complementary strengths of diverse hypotheses. Multiple learners can be combined using non-trainable algebraic combiners or trainable meta-combiners.

Explanation

Individual learning models often suffer from statistical, computational, and representational limitations. Combining multiple learners mitigates these limitations by expanding the effective hypothesis space and reducing prediction variance. Algebraic schemes aggregate base model outputs directly.

Example

Simple Majority Voting: For classification tasks, each base classifier casts one vote for a candidate class label, and the ensemble predicts the class receiving the absolute or plurality majority of votes.

Algorithm / Steps

1. Initialize equal observation weights across all N training samples: w_i^(1) = 1 / N.
2. In each boosting iteration m = 1 to M, train base learner h_m(x) using sample weights w_i^(m).
3. Compute the weighted classification error: epsilon_m = Sum of w_i^(m) for all misclassified instances divided by the sum of all sample weights.
4. Calculate the model importance weight: alpha_m = (1 / 2) * ln((1 - epsilon_m) / epsilon_m).
5. Update instance weights: w_i^(m+1) = w_i^(m) * exp(-alpha_m * y_i * h_m(x_i)) and normalize so weights sum to one.
6. The final composite ensemble decision is H(x) = sign(Sum from m=1 to M of (alpha_m * h_m(x))).

Key points

- Combining multiple learners expands the effective hypothesis space and reduces prediction variance.
- Algebraic combiners aggregate base model outputs directly, e.g., simple majority voting, weighted voting, soft voting, rank-based voting.

- Weighted voting assigns a reliability weight to each classifier, scaling its vote by alpha_l.
- AdaBoost implements a weighted voting scheme where each weak learner contributes to the final decision with weight alpha_m.

Ensemble Learning

Definition

Combining multiple individual learners into a combined predictive model leverages complementary strengths of diverse hypotheses. Ensemble learning is a machine learning paradigm where multiple base learners (often called weak learners) are systematically generated and combined to solve a common predictive problem, producing a strong ensemble learner.

Explanation

Ensemble learning mitigates statistical, computational, and representational limitations of single models by expanding the effective hypothesis space and reducing prediction variance. It relies on diversity and independence among base learners; if each learner has an error rate below 0.5 and they are statistically independent, the ensemble error probability decreases as the number of learners increases (Condorcet's Jury Theorem). Ensemble methods alter the bias–variance decomposition: averaging methods primarily reduce variance without increasing bias, whereas boosting iteratively reduces bias by focusing on previously misclassified instances.

Algorithm / Steps

1. Create L bootstrap samples D_l of size N by sampling with replacement from the training dataset D.
2. Train a base learner h_l(x) on each bootstrap sample in parallel.
3. Combine base predictions: for classification use plurality voting; for regression average predictions: y(x) = (1 / L)Sum_l=1^L h_l(x).
4. Initialize equal observation weights w_i^(1) = 1/N for all N training samples.
5. For each boosting iteration m = 1 to M, train base learner h_m(x) using sample weights w_i^(m).
6. Compute the weighted classification error epsilon_m = fracSum w_i^(m) for misclassified instancesSum w_i^(m).
7. Calculate the model importance weight alpha_m = (1 / 2)ln!((1-epsilon_m / epsilon_m)).
8. Update instance weights: w_i^(m+1) = w_i^(m)exp(-alpha_m y_i h_m(x_i)) and normalize so weights sum to one.
9. The final composite ensemble decision is H(x) = sign!(Sum_m=1^Malpha_m h_m(x)).

Key points

- Ensemble learning combines multiple learners to leverage complementary strengths and reduce prediction variance.
- Diversity among base learners—achieved by varying data, features, or algorithms—is essential for strong ensemble performance.
- Bagging reduces variance by training learners on bootstrap samples and aggregating predictions.
- Boosting reduces bias by sequentially focusing on hard-to-classify instances and weighting learners accordingly.

bagging, boosting, stacking

Definition

Ensemble techniques combine multiple base learners to produce a single predictive model that typically outperforms any individual learner. Bagging, boosting, and stacking are three common ensemble strategies. Bagging trains many base learners in parallel on different bootstrap samples of the data. Boosting trains base learners sequentially, each focusing on the mistakes of the previous ones. Stacking trains several heterogeneous base learners and then uses a meta-learner to blend their predictions.

Explanation

Ensemble methods reduce error by exploiting diversity among base learners. According to Condorcet's Jury Theorem, if the base classifiers are statistically independent and each has an error rate below 0.5, the ensemble error decreases as the number of classifiers increases. Diversity is introduced by varying training data subsets, feature subsets, or learning algorithms. Bagging primarily reduces model variance without increasing bias, while boosting iteratively reduces bias by concentrating on hard examples.

Algorithm / Steps

1. For a new instance x, aggregate predictions:
   - Classification – plurality voting.
2. Use the 36.8 % of data not selected in a bootstrap sample as an out-of-bag (OOB) validation set to estimate generalization performance.
3. AdaBoost turns weak learners into a strong learner by reducing bias.
4. Train each base learner on the full training set.
5. Use k-fold cross-validation to generate out-of-fold predictions (meta-features) for every training instance.
6. Train a meta-learner on these meta-features to learn an optimal blending strategy for predicting the target variable.

Key points

- Ensembles rely on diversity and independence to lower error.
- Bagging reduces variance by averaging predictions from parallel learners.

- Boosting reduces bias by iteratively re-weighting misclassified instances.
- Stacking blends heterogeneous learners through a meta-learner trained on out-of-fold predictions.
- Out-of-bag validation in bagging provides an efficient way to estimate generalization error.

Unsupervised learning: K-means

Definition

K-Means is a centroid-based partitioning clustering algorithm that divides N data points into K non-overlapping clusters C = C1, C2, ..., CK. Each cluster is represented by its geometric centroid mu_k.

Explanation

K-Means minimizes the within-cluster sum of squares (WCSS), also known as cluster inertia: J = Sigma_k=1^K Sigma_x_iinC_k ||x_i - mu_k||^2.

Algorithm / Steps

1. Assignment Step: Assign each data point x_i to the nearest centroid based on Euclidean distance: c_i = argmin_k ||x_i - mu_k||^2.
2. Update Step: Recompute each cluster centroid as the arithmetic mean of all data points assigned to that cluster: mu_k = (1 / |C_k|) * Sigma_x_iinC_k x_i.
3. Convergence: Repeat assignment and update steps iteratively until cluster assignments no longer change or centroid displacement falls below a predefined tolerance threshold.

Key points

- K-Means converges in polynomial time to a local minimum of the objective function.
- It scales linearly as O(N * K * d * I) where I is iteration count.
- Requires specifying K in advance, often evaluated using the Elbow method or Silhouette analysis.
- Assumes spherical, isotropic clusters of equal size.

Instance Based Learning: KNN

Instance-based learning, also known as lazy learning or memory-based learning, delays generalization and model induction until a query instance is presented for prediction. Instead of constructing an explicit global parametric function during training, the algorithm stores all training instances in memory and performs local computations on demand.

The K-Nearest Neighbors (KNN) algorithm operates as follows:

1. Distance Metric: Given a query instance x_q, KNN computes the distance between x_q and all stored training instances x_i using Euclidean distance d(x_q, x_i) = sqrt(Sum from j=1 to d of (x_q,j - x_i,j)^2) or Manhattan distance d(x_q, x_i) = Sum from j=1 to d of |x_q,j - x_i,j|.

AcademicsGen — source: NPTEL — CC BY-SA 4.0

2. Neighbor Selection: The algorithm identifies the K training instances closest to x_q.
3. Prediction: For classification, KNN assigns x_q to the majority class among the K nearest neighbors: y_hat = argmax_c Sum from i=1 to K of I(y_i = c). For regression, KNN outputs the arithmetic mean of target values: y_hat = (1 / K) * Sum from i=1 to K of y_i.
4. Distance-Weighted KNN: To reduce the influence of distant neighbors within the neighborhood, each neighbor's contribution is scaled inversely by its distance: w_i = 1 / (d(x_q, x_i)^2 + epsilon).

KNN makes zero parametric assumptions about the underlying data distribution. Small values of K lead to low bias but high variance, whereas large values of K increase bias. KNN is sensitive to feature scaling and suffers from the curse of dimensionality, where distance metrics lose discriminative power in high-dimensional spaces.

Gaussian mixture models and Expectation maximization

Gaussian mixture models and Expectation Maximization are key techniques in unsupervised learning.

Definition

A Gaussian Mixture Model (GMM) is a probabilistic generative model that represents the overall data distribution as a linear superposition of K distinct multivariate Gaussian component densities: p(x) = Sigma_k=1^K (pi_k * N(x | mu_k, Sigma_k)) where pi_k denotes the mixture weight of component k, satisfying 0 <= pi_k <= 1 and Sigma_k=1^K pi_k = 1, and each component is parameterized by its mean vector mu_k and covariance matrix Sigma_k.

Explanation

Unlike K-Means, which performs hard cluster assignments, GMM performs soft probabilistic clustering by computing responsibilities gamma(z_ik), representing the posterior probability that data point x_i was generated by component k: gamma(z_ik) = (pi_k * N(x_i | mu_k, Sigma_k)) / Sigma_j=1^K (pi_j * N(x_i | mu_j, Sigma_j)) The soft assignment allows each data point to belong to multiple clusters with varying probabilities.

Algorithm / Steps

1. **E-Step (Expectation)** Using current parameter estimates theta^(t) = pi_k, mu_k, Sigma_k, compute the posterior responsibilities gamma(z_ik) for every data point x_i and component k.
2. **M-Step (Maximization)** Update model parameters using the computed responsibilities:
   - Effective count: N_k = Sigma_i=1^N gamma(z_ik)
   - Updated mixture weights: pi_k^(t+1) = N_k / N
   - Updated means: mu_k^(t+1) = (1 / N_k) * Sigma_i=1^N (gamma(z_ik) * x_i)

- Updated covariances: Sigma_k^(t+1) = (1 / N_k) * Sigma_i=1^N (gamma(z_ik) * (x_i - mu_k^(t+1)) * (x_i - mu_k^(t+1))^T)

3. **Convergence** Evaluate the data log-likelihood ln(p(X | theta)) and repeat E and M steps until the increase in log-likelihood falls below a convergence tolerance. The EM algorithm guarantees monotonic non-decreasing improvements in data log-likelihood toward a local maximum.

Key points

- GMM models data as a weighted sum of multivariate Gaussian components.
- Responsibilities provide soft cluster assignments, unlike hard assignments in K-Means.
- EM iteratively performs an E-step to compute responsibilities and an M-step to update parameters.
- The algorithm ensures monotonic improvement of the data log-likelihood.

Source and licence

This PDF is adapted from **Artificial Intelligence and Machine Learning (Ensemble Techniques and Unsupervised Learning)** (https://nptel.ac.in/courses/106105077), accessed .

Source licence: **CC BY-SA 4.0** — https://creativecommons.org/licenses/by-sa/4.0/

Changes made by AcademicsGen: the source text was rephrased, reorganised and grammar-corrected into study material; nothing added.

This PDF is shared under the same licence (CC BY-SA 4.0).